

C++ 面试常考点汇总

杭电 109 期 C++ 寇文宇

前言: 所有考点均是本人根据比特的课程以及刷的 60 篇+面经后的总结, 用于本人的校招的总复习, 回答仅供参考, 请理性阅读, 希望这份文件能帮助你复习一些已经遗忘的知识. 背八股文不是校招的终点,, 没掌握的知识终究会在未来的某一天化为冰雹打在我们的脸上. 一起加油, 最后祝愿大家能拿到心仪的 offer, 去到心仪的城市, 成为技术大牛!!!

自我介绍部分:

老师您好, 我叫寇文宇, 是杭州电子科技大学自动化专业的一名本科生, 将在 2025 年 6 月毕业. 虽然我是自动化专业, 但是我对计算机有着浓厚的兴趣, 在校期间我通过蹭课和上网课, 看资料自学的方式系统的学习了 C/C++, 基本的数据结构, 常见的算法思想, Linux 系统, Linux 网络以及数据库相关的知识. 我通过做笔记, 写博客的方式记录自己的学习历程, 现在我已经发布超 140 篇原创的技术博客, 收获了一万多粉丝, 并且多篇文章被各大开源社区给收录了. 在工作经历方面, 我在腾讯 QQ 部门的基础架构中心实习了小半年, 主要是做用户资料和关系链相关的工作. 在实习期间完成了两个很有价值的项目, 一个是 42 亿流水数据中心的改造. 另外一个为用户资料门户的构建. 由于这两个项目涉及到诸多细节和难点, 这里就不一一赘述了, 有兴趣的话你后面可以追问我. 我也是利用工作闲暇的时间, 自学了一些业内常用的开源组件, 如 Kafka, ES, redis, docker 以及一些业内常用的数据库, 如 postgresql, click house, mongodb 等. 我的整个学习历程和实习经历, 是为我成为一名优秀的后端开发工程师, 打下了坚实的基础, 所以对于贵公司的这个岗位, 我抱有绝对的信心, 也对公司后台使用的技术, 抱有极大的兴趣, 希望通过此次面试能够对我们双方有一个更深刻的了解

最后反问技术问题部分:

1. 公司常用的技术栈是什么? (主要使用什么语言?)
2. 团队目前主要是在做什么内容?
3. 贵部门现在主要是在开发新的功能还是在维护原有的功能?
4. 一个项目从规划到最后完成任务的基本流程是怎样的? 我能参与多少?
5. 贵公司有标准的开发环境吗? 平时怎样测试代码?
6. 你们是怎样准备故障恢复的?
7. 你希望在我们应届生身上看到什么特质?
8. 你是怎样学习后端的? 能否给我一些经验?

最后反问 HR 问题部分:

1. 新人培训是具体是怎样进行的?
2. 部门的具体分工是怎样的? 一个部门或一个团队大约有多少人?
3. 团队内或者是团队之间是如何进行沟通的? 开会吗?
4. 如果遇见意见分歧, 应该怎样处理?
5. 一个项目从规划到最后完成任务的基本流程是怎样的? 我能参与多少?
6. 公司有技术交流会吗? 我能学到什么?
7. 公司有要求写个人的工作报告吗? 写 PPT 这种?
8. 公司留用实习生的标准是什么?
9. 您认为考核这个岗位员工的最重要指标有哪些?
10. 有员工宿舍吗? 离公司远不远? 租房有补贴吗?

HR 面:

1. **自我介绍:** 我叫寇文字,是杭州电子科技大学的一名大三学生, 我准备从四个方面全面的介绍我自己.

A.第一点: 我最深入研究的知识领域以及正在学习的方向: 我比较擅长的领域是 C++ 开发及其底层的一些原理和 Linux 操作系统系统以及基础的数据结构.

B.第三点: 我最强的技能, 能坚持下来做一件事,一旦决定了一件事就会坚持到底

C.第二点: 我做过最成功的事情, 一件是坚持写博客记录学习历程,目前已经是发布了超 110 篇原创的技术博客,在此期间帮助了很多有技术问题的同学,让我很有自豪感,也遇见了一些志同道合的同窗让我拓宽了视野,在写博客期间也巩固了我的知识体系, 另一件是坚持健身,让自己从一个身体健康比较差的状态下变成了一个身心健康并且自信,乐观的人

D.第四点: 我性格上最大的优点: 外向,善于沟通交流,能很快的融入团队, 乐观,我抗压能力比较强,能在压力下正常工作

2. 你最大的缺点是什么?

我头比较铁, 我平时在专研一个未知领域的技术时, 有时会出现不把一个模块搞懂就不吃饭的情况.

3. 你对加班的看法?

我觉得加班可以分为两种情况,一是业务出现了突发的状况,或者一个项目需要赶快上线等正常的加班需求,我肯定是义不容辞的,毕竟我目前没有家庭负担,可以全身心投入工作. 第二点是我不太能接受的一点,就是在正常做完一天的工作之后,没有特殊的任务时,为了彰显自己

工作比较努力而进行的无效加班

4. 你对薪资的要求?

首先,我对工资没有硬性的要求, 第一点,我认为贵公司发展了这么多年,一定是有自己的一套合理的薪资体系,我相信公司会给我一个合理的安排.第二点,我现在是实习生,赚钱并不是首要任务,我觉得更重要的是赶快熟悉公司的业务, 努力提升自己的技术水平, 最后有机会能转正留在贵公司.

5. 你的职业规划是什么?

我先说一个我的短期规划和长期规划吧,短期规划的话,就是想尽快熟悉公司的业务,提高我自己的能力,能在实习的最后获得转正的机会,长期来看的话, 我准备在技术领域有所作为,我经过 1~3 年的沉淀学习能让我掌握完备的领域知识,以及优秀的代码能力,并且能为将来成为高级 C++ 工程师打下基础. 工作 3~5 年我希望自己能有深度的思考能力以及技术的鉴别能力,在工作中能独当一面,并且能指导新手完成任务. 在工作的 5~7 年,我争取能被评为高级 C++ 工程师,并且我希望我能具备一个能当管理者的基础

6. 如果本次面试没有录用你,你怎么打算?

我觉得找工作这件事本身就是一个双向选择的过程,如果现在的我没有达到公司招聘人的要求,我会更加的鞭策自己去努力,并且也希望在未来某一天能和贵公司进行合作

7. 如果被录取后,工作一段时间发现你不适合这个职位,应该怎么办?

我觉得有两种情况, 一是我的确热爱这个职业, 那么我就要不断学习,向领导,同事学习业务

知识和专业技能, 力争减少差距. 二是我的确也觉得这个工作不适合我,我做起来很痛苦,那么我会毫不犹豫的辞掉工作换个职业,这对于我和公司的发展都有好处

8. 如果你工作出现错误,给公司造成经济损失,应该怎么办?

首先肯定是要想办法去弥补或挽救经济损失,如果我实在无能力负责,希望单位帮助我解决. 其次,认清责任,如果是我的责任,我甘愿受罚,绝不会逃避.如果是团队中其他人出现的错误我也不会幸灾乐祸,作为一个团队我将给予我最大的帮助

最后,总结经验教训, 避免在今后的工作中发生类似的错误. 检讨自己的工作方式是否有问题,分析问题时是否考虑周全.

9. 你对我们公司了解多少?

首先,我是从 21 年的一篇叫“数字反哺”的文章开始关注贵公司的,那个时候正是疫情期间,很多老年人是不知道怎样在微信里打开健康码,通过将小程序添加到桌面的方式,方便了老年人快速打开健康码,这种做具有社会价值的工作也比较符合我的个人理念,包括向后来贵公司利用 AI 技术来保护珍惜动物如雪豹和候鸟,以及方便人们生活的刷掌支付,更让我坚信了想要来腾讯的信念. 其次,贵公司的企业文化也是吸引我来腾讯的原因之一,正值,进取,协作,创造,我记得有句话叫无为便是过,要勇于突破有担当,这句话是深深的印进了我心里

C 语言部分:

1. 说说 static 关键字的几个用处

A. 修饰局部变量

正常的局部变量在出了作用域后就会销毁,使用 static 修饰后的局部变量生命

周期变长,当整个程序走完后才会销毁(作用域不变,出了作用域依旧无法被识别)

B. 修饰全局变量

正常的全局变量在任何.C 文件中都可见,static 修饰后,它的链接属性

由外部链接变成内部链接,使得这个全局变量只能在本.c 文件可见

C. 修饰函数

用 static 修饰函数后,会改变函数的链接属性,由外部链接变成内部

链接,导致此函数只能在本.C 文件中可见,其他文件不可见

2. 谈谈你对指针的理解

A. 先谈对指针的宏观理解

首先, 指针就是内存当中的一个编号, 它是变量的地址,

也就是说指针即为地址,存储这个地址的变量叫做指针变量

B. 再谈指针的大致分类

指针可以分为一级指针,二级指针,数组指针,函数指针,

还有野指针等,学长您看看要不要挑选一个来详细谈谈?

C. 面试官有细谈要求再展现知识能力

3. 源码, 反码, 补码的关系

A. 正数的源反补码相同

B. 负数的反码是源码取反(除符号位), 补码是反码加一

4. 什么是大小端? 说说你的理解 (腾讯一面)

A. 字面的定义

大端存储模式: 指数据的低位保存到内存的高地址上,而数据的高位保存在内存的低地址上

小端存储模式: 指数据的低位保存到内存的低地址上,而数据的高位保存在内存的高地址上



B. 设计一个程序判断当前机器是大端还是小端

```
int main()
{
    //写一段代码告诉我们当前机器的字节序是什么
    int a = 1;
    char* p = (char*)&a;
    if (*p == 1)
    {
        printf("小端\n");
    }
    else
    {
        printf("大端\n");
    }
}
```

```
union
{
    int i;
    char c;
}un;
un.i = 1;
return un.c;
```

5. 谈一谈结构体内存对齐规则, 以及为什么要有内存对齐?

(1) 先回答内存对齐规则

- A. 第一个成员在与结构体变量偏移量为 0 的位置
- B. 其他成员变量要对齐到对齐数的整数倍位置
- C. 对齐数 = min(编译器默认的对齐数 , 该成员变量大小), VS 下默认对齐数为 8
- D. 结构体总大小为每一个成员变量的最大对齐数的整数倍
- E. 修改默认对齐数的方法: #pragma pack(8) , 设置默认对齐数为 8

(2) 再回答为什么要有内存对齐

A. 为了平台可移植,

不是所有的硬件都能访问任意地址上的任意数据, 可能只能在 4 的倍数的地址处取数据

B. 为了性能

若内存不对齐, 原本只需要访问一次的数据现在可能要访问两次才能拿全

6. 谈一谈位段, 联合体, 枚举的区别

A. 位段的定义

位段的声明和结构体类似, 位段的成员必须是 int、

unsigned int 或 signed int (char 也可以), 位段的成员

名后面有一个冒号和一个数字, 数字代表这个成员要占的字节数

```
struct A
{
    int _a:2;
    int _b:5;
    int _c:10;
    int _d:30;
};
```

B. 联合体的定义

联合也是一种特殊的自定义类型, 使用 union 定义

这种类型定义的变量也包含一系列的成员, 特征是这些成员公用同一块空间

```
union Un
{
    int i;
    char c;
};
```

C. 枚举的定义

枚举也是一种类型, 使用 enum 定义

{ } 中的内容是枚举类型的可能取值, 也叫枚举常量

这些取值都有值, 默认从 0 开始, 依次递增一

```
enum Sex//性别
{
    MALE,
    FEMALE,
    SECRET
};

enum Color//颜色
{
    RED=1,
    GREEN=2,
    BLUE=4
};
```

7. 谈一谈程序从编译到运行的整个过程（美团二面）

- A. 预处理阶段: 头文件展开, 去掉注释, 条件编译, 宏替换, 形成 .i 文件 (还是 C 语言)
- B. 编译阶段: 将 C 语言翻译为汇编语言, 形成 .s 文件 (此时是汇编代码)
- C. 汇编阶段: 将汇编代码变成可重定位二进制文件, 形成 .o 文件
- D. 链接阶段: .o 文件 + 系统库 形成可执行程序

8. 谈一下 extern 关键字的作用（腾讯一面）

Extern 关键字用于声明一个变量或者函数

指明它是在其他文件中定义的,提醒编译器去其他文件中找定义

9. 谈一下 volatile 关键字的作用（腾讯一面）

Volatile 关键字用于告诉编译器,当前修饰的变量可能会被意外修改,

编译器不能直接到寄存器中拿这个变量而是必须去内存中拿这个变量

10. #define 和 const int 定义宏的区别?你更倾向于哪种?(小米一面)

A. 先说区别

1. #define 是预处理指令,它在预处理阶段就进行了文本替换,

const int 是在编译.运行阶段展开

2. `#define` 只是简单的文本替换,不会对类型安全进行检查, `const` 会进行类型检查

3. `#define` 可能会导致调试困难以及可读性不佳,

而 `const int` 定义的是真正的变量, 容易调试

B. 再说自己的编码习惯

11. 谈一谈指针常量和常量指针 (滴滴一面)

A. 常量指针

常量指针是这样声明的: `const int* p` 或 `int const * p`

它代表只能够读取 `p` 指向的数据,但不能修改 `p` 指向的数据

但是它可以修改 `p` 的指向

B. 指针常量

指针常量是这样声明的: `int* const p = &a`

它代表这个指针的指向不能被修改

但是这个指针指向的内容可以被修改

12. 谈一下 `malloc`, `calloc` 和 `realloc` 的区别

A. `Malloc` 和 `calloc` 的区别

`malloc` 申请空间不会初始化, `calloc` 会将空间全部初始化为 0

`Malloc` 传参要申请的总字节数, 而 `calloc` 传参元素个数和每个元素大小

B. `Realloc` 和它们两个的区别

`Realloc` 是对已经申请好的空间做大小调整, 传参期望的总字节数

13. 能否讲一下 malloc 的原理?

- A. 当申请的空间小于 128KB 时会先去空闲链表管理的空闲内存中查看,若空闲的内存能满足所需大小,则直接分配该内存.若空闲内存不足,则需要调用系统调用扩大空间来满足需求
- B. 当申请的空间大于 128KB 时,会直接调用系统调用来开辟空间
- C. 调用 malloc 时,只是分配了对应的虚拟地址空间。只有当访问该部分内存时才会真正分配物理内存并将物理内存和虚拟内存建立映射关系,并且根据实际使用多少内存分配多少物理内存

14. 按行和按列遍历二维数组在性能上的差异? (快手二面)

按行遍历访问的内存是连续的, 可以提高内存访问效率,减少缓存未命中的情况,
而按列遍历都导致内存访问的不连续性, 效率比较低

15. 手 撕 strcpy, strcat, strcmp, strstr, memset, memcpy, memmove

C++ 部分:

16. C 语言和 C++ 的区别?

- A. C 语言是面向过程的语言, 注重的是一步一步的调用函数解决问题, C++ 是面向对象的语言, 会将问题分解为若干个对象, 再围绕着对象去解决问题
- B. C++ 的标准库比 C 语言的更强大, 引入了诸如 STL 库, 智能指针, 模板等新的内容, 使得程序员编写代码变得更简洁了

17. 谈一谈函数重载和底层的原理

A. 先谈什么是函数重载

C++ 允许在同一作用域定义多个函数名相同的函数,这些同名函数的

形参列表(参数个数 或 参数类型 或 参数顺序)必须不同

B. 再谈底层原理

首先要知道程序的各个阶段在做什么, 在经过了预处理,编译和汇编阶段后,会形成 .O 文件.

在链接阶段, C 语言中的符号表是将函数名和函数地址——对应起来了, 所以一旦出现多个相同名字的函数, 就会有多个相同的函数地址, 就会分不清应该调用哪个

而 C++ 的多了一个函数名修饰规则, 它会把函数名和参数都带到符号表中, 然后汇编 call 函数地址时就会跳到不同的函数

18. 谈一下你对引用的理解

A. 先谈宏观的定义

引用就是给一个已存在变量取别名, 编译器不会为引用变量开辟空间, 它和它引用的变量共用同一份空间

引用必须在定义时就初始化, 一个变量可以有多个引用, 但一旦引用一个实体后, 就不能再引用其他实体

B. 最经典的问题, 指针和引用的区别,从两个角度回答问题

(1) 语法使用角度:

引用没有额外开辟空间, 而指针变量会开辟 4/8 字节的空间

引用在定义时必须初始化, 而指针没有要求

一旦引用一个实体,就不能再引用另一个, 而指针可以指向任意同类型的实体

引用不存在空引用, 但是指针存在空指针. (引用的内容可以指向空,但内容本身不能为空)

引用的类型大小是被引用的变量的大小, 而指针的大小就是固定的 4/8 字节

(2) 底层实现角度:

我查看过引用和指针的汇编代码,发现它们其实是一样的,也就是说引用的底层是用指针实现的

19. 谈谈你对内联函数的理解 (深信服一面)

用 inline 修饰的函数叫做内联函数, 在编译时会将调用内联函数的地方展开, 没有的函数建立的创建栈帧的开销, 可以提高程序运行的效率. 内联函数只是对编译器的一个建议, 不是所有被 inline 修饰的函数都会被展开

20. 简述一下早期的 auto 和 C++11 后的 auto 的区别

早期的 auto 的含义是: 使用 auto 修饰的变量, 是具有自动存储器的局部变量

C++11 后的 auto 被用作自动类型推导

21. 说一下 struct 和 class 的区别

C++把 struct 升级成为了类, struct 的访问权限默认为 public,

而 class 的访问权限默认为 private.

在继承中, struct 的默认继承方式是 public, 而 class 的默认继承方式是 private

纯 C 语言中, struct 只是数据的集合, 而在 C++ 中, class 是数据加方法的集合

22. 谈一谈你对封装的理解

封装就是将数据和处理数据的方法统一进行整合, 并且将对象的属性和实现细节给隐藏了起来, 仅仅对外公开部分接口来和对象进行交互

23. 一个类的大小应该怎样计算?成员函数放在哪儿了?

A. 一个类的大小就是该类中的成员变量之和(算上内存对齐),

空类(或只有成员函数的类)要占一个字节

B. 成员函数放在了公共代码段, 在编译链接时会根据函数名去公共代码段找到函数的地址

24. This 指针存储在哪里?This 指针可以为空吗? (360 一面)

A. this 指针是形参,应该被存放在栈上, 但编译器也可能把 this 放在 ECX 寄存器中

B. this 指针可以为空, 若函数内部不会使用到 this 指向当前当前对象就不会有问题,

如果在函数内部访问了成员变量,此时 this 为空就会对空指针解引用

25. 在析构函数写 delete this 会出现什么情况?

delete this 会调用 this 的析构函数, 析构函数又会 delete this. 会死循环. 不能在实际代码中写这样的语句

26. 请列举出不能运算符重载的操作符

一共五个: `. * :: sizeof ? : .`

27. 请说明运算符重载前置++和后置++的区别

前置++和后置++都是一元运算符, C++规定后置++重载时要多增加一个 int 类型的参数

28. 哪些成员必须放在初始化列表初始化?

三种成员: const 成员, 引用成员, 自定义类型成员(并且该类没有默认构造函数)

特别的是, 初始化列表不能初始化静态成员, 所以 static const int 类型的成员不能放在初始化列表中

29. 给我讲讲 explicit 关键字的用处

A. 先讲背景

对于单个参数或除了第一个参数无默认值,其余参数都有默认值的构造函数具有隐式类型转换的作用, 比如 `string str = "abcdef"`, 这里编译器会用 "abcdef" 字符串构建一个无名对象,再将这个对象给 str 赋值.

B. 再讲 explicit 的作用

在构造函数前面加上 explicit 关键字可以阻止这种转换发生

30. 谈一谈类中 static 成员的特性

A. 先谈成员变量

(1) 静态的成员变量只能在类外初始化

(2) 静态成员变量是所有类对象共享的,只有一份

(3) 静态成员变量可以用类域::静态成员或对象.静态成员访问, 且受访问限定符的限制

B. 再谈成员函数

(1) 静态成员函数没有 this 指针, 不能访问类中的非静态成员和函数

(2) 静态成员函数也能直接用类域访问,或用对象访问

31. 谈一下 new/delete 和 malloc/free 的区别

(1) Malloc 和 free 是函数,而 new 和 delete 是操作符

(2) Malloc 申请空间要显示传需要申请的大小,而 new 只用传类型和个数

(3) Free 时只需传入要释放的指针, 而 delete 可能要加上[]

(4) Malloc 失败后返回 NULL,而 new 失败后会抛异常

(5) Malloc 返回值是 void* 使用时要强转, 而 new 不用

(6) Malloc 申请的空间既不会初始化也不会调用自定义类型的构造函数

而 new 申请空间不仅会初始化,还会调用自定义类型的构造函数

(7) Free 释放空间不会调用自定义类型的析构函数

而 delete 释放空间会自动调用自定义类型的析构函数

32. 能聊一下 new/delete 的底层原理吗?

New 底层调用 operator new 全局函数来申请空间

Delete 底层调用 operator delete 全局函数来释放空间

而 operator new 底层其实是调用 malloc 来申请空间的

Operator delete 底层是调用 free 来释放空间的

33. 什么是内存泄漏? (腾讯一面)

内存泄漏是指:因为错误或者疏忽导致程序没有释放已经不再使用的资源,从而失去对这份资源的控制,使得可用的资源越来越少,比方说我知道的内存泄漏对象有:堆空间,套接字,文件描述符

34. 什么是泛型编程?给我讲一下模板的原理

- A. 泛型编程就是编写与类型无关的通用代码,是代码复用的一种手段
- B. 模板将本应该我们重复做的事情交给了编译器,在编译阶段,当编译器推演出你想要处理的变量的具体类型时,比如说 double 类型,编译器会专门生成一份处理 double 类型的代码

35. 为什么要有模板的特化?特化是怎样实现的?

- A. 因为在特定场景下,一些特殊的类型可能会得到错误的结果
此时需要在原模板的基础上,针对特殊类型做特殊化的处理方式

B. 先说函数模板的特化

1. 必须要先有一个基础的函数模板
2. 再写上关键 template 后面加上 <>
3. 函数名后跟一对尖括号 <>,并在尖括号内指定要特化的类型
4. 函数形参个数要和基础模板相同,参数类型写指定的类型

```

template<class T>
bool Less(T left, T right)
{
    return left < right;
}

// 对Less函数模板进行特化
template<>
bool Less<Date*>(Date* left, Date* right)
{
    return *left < *right;
}

```

C. 再说类模板的特化

类模板特化分为全特化和偏特化, 全特化就是将模板的所有参数都确定化, 其形式和函数模板的特化类似, 而偏特化又分为两种表现形式, 部分特化和对参数的进一步限制, 学长你看看需要细聊哪一点?

D. 偏特化的细节

部分特化: 将模板参数列表中的参数一部分特化, 其实现方式和全特化类似

对参数的进一步限制: 即对模板参数做进一步条件限制, 比如将两个参数偏特化为指针类型, 或将两个参数偏特化为引用类型

36. 手撕一个 string 类

宗旨是代码可通过编译, 代码简单, 容易实现即可, `char* str, int size, int capacity`

先实现, 无参构造, 带参构造, 拷贝构造, 析构函数, `operator=`, `operator[]`

如果写的比较快可以再写上 C++11 的移动构造和移动赋值

37. 谈一下 vector 的容量问题 (腾讯一面)

A. Vector 是如何进行扩容的?

当向 vector 中插入元素时,有效元素个数 size 和空间容量大小 capacity 相同时,vector 会触发扩容机制, 会开辟新空间,拷贝元素,再释放旧空间

B. 扩容会导致效率低下, 如何避免动态扩容?

如果在插入元素前,可以预估 vector 中存储元素的个数,可使用 reserve 函数提前把底层的空间开辟好

C. Windows 下和 Linux 下的扩容机制有何不同?

VS 下选择 1.5 倍的方式扩容, g++ 下选择 2 倍的方式扩容

38. 谈一下 vector 的迭代器失效问题 (迅雷一面)

迭代器失效,实际就是迭代器底层对应指针所指向的空间失效了,

如果继续使用已经失效的迭代器,程序可能会崩溃。

可能引起迭代器失效的操作:

- (1) 引起底层空间改变都有可能让迭代器失效, resize, reserve, insert, push_back 等
- (2) Erase 操作:删除一个元素后,后面的元素会向前移动,这不会有问题,但若删除的是最后一个元素,pos 就指向 end 位置,end 位置没有元素.

如何避免出现迭代器失效问题?

在 insert/erase 一个 pos 位置后,默认 pos 已经失效, 不要再访问它,或在使用前重新赋值

39. Vector 和 list 的区别?

A. Vector 是顺序容器, 当底层的 size 等于 capacity 时会进行扩容, 开辟一份新空间并且拷贝之前的数据, list 是双向链表, 不会有扩容的问题

B. Vector 的底层是连续的空间, 而 list 的底层是不连续的空间

- C. Vector 能使用随机迭代器高效的访问数据, 而 list 只能用双向迭代器来访问数据
- D. Vector 的插入和删除可能会移动或拷贝数据, 效率较低, 而 list 插入和删除数据效率较高

40. 手撕一个链表

- A. 首先先封装一个 ListNode 类,写上构造函数,不要写模板
- B. 再封装一个迭代器类, 要实现构造函数, operator*,operator->,operator!=,operator++和 operator--,暂时只写非 const 的迭代器
- C. List 类中要写无参构造, 析构函数, begin 和 end 函数,insert 和 erase 函数,再复用 insert 写一个 push_back,再用 push_back 写一个拷贝构造

41. 谈一谈你对继承的理解

继承是代码复用的一种手段, 它允许在基于已存在的类上做拓展, 增加新的功能, 体现出了面向对象程序设计的一种层次结构, 继承还是实现多态的基础, 是很重要的手段. C++中的一些设计模式都是基于继承和多态的.

42. 继承基类成员的访问方式变化

Public 继承: 基类成员的原先访问属性不变

Protected 继承: 基类的 public 成员变成 protected 成员, 其余不变

Private 继承: 基类的所有成员都是 private

基类的 private 成员在子类中不可见

43. 说一下函数重载和函数隐藏的区别

首先, 构成函数重载必须在同一作用域中,有同名函数

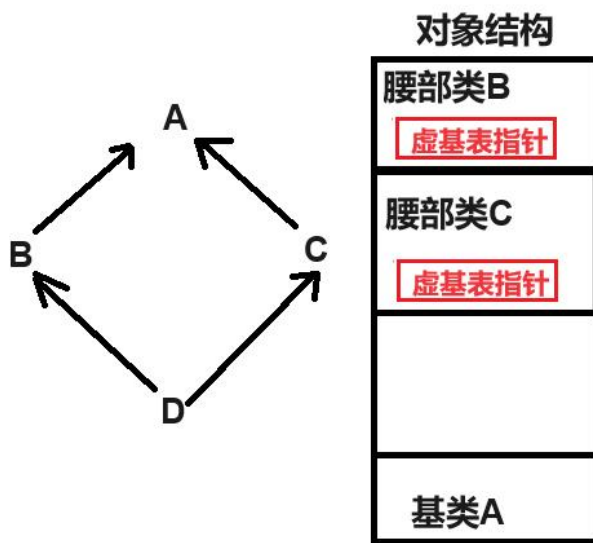
而函数隐藏则是在继承体系中,基类和子类中有同名函数,这两个函数就构成隐藏

44. 谈一下派生类的默认成员函数的特性

- A. 创建对象时, 先构造父类再构造子类, 先析构子类再析构父类
- B. 拷贝构造和 operator= 都必须去调用在基类对应的函数(指定类域调用)
- C. 子类的析构和父类的析构, 构成隐藏, 子类析构不用显示调用父类析构

45. 怎样解决菱形继承的数据冗余和二义性问题?

- A. 首先, 数据的二义性问题可以通过指定腰部类的类域来解决, 但这种方法不能解决数据冗余
- B. 虚拟继承可以解决这两个问题, 在腰部类继承时加上关键字 virtual, 虚拟继承改变了腰部类的结构, 基类现在就只有一份了, 同时属于两个腰部类, 腰部类中会多出一个虚基表指针, 指向虚基表, 虚基表中存储了腰部类到基类的偏移量



46. 继承和组合(is-a 和 has-a)的区别?哪些场景用哪个?

- A. 继承是一种 is-a 的关系, 每一个派生类对象都是一个基类对象, 它破坏的封装, 并且父类和子类有很强的耦合度,
- B. 组合是一种 has-a 的关系, 假设 B 组合了 A, 每一个 B 对象都有一个 A 对象, 组合类之间没有很强的关联性, 耦合度低, 应该优先使用组合而不是继承

47. 谈一谈你对多态的理解

A. 先谈多态的定义:

多态就是多种形态, 是指在继承体系中, 不同对象去调用同一个函数会出现不一样的结果

B. 再谈满足多态的条件:

1. 必须是在继承体系中
2. 必须通过基类的指针或引用调用虚函数
3. 被调用的函数必须是虚函数, 且派生类必须对基类的虚函数进行重写

48. 谈一谈你对函数覆盖(重写)的理解

若派生类中有一个和基类完全相同的虚函数(返回值,函数名,参数列表都相同), 则称子类的虚函数覆盖(重写)了基类的虚函数. 函数覆盖有两个特例:

1. 子类虚函数不加 virtual 也构成覆盖.
2. 返回值可以不同, 基类虚函数可返回基类对象的指针或引用, 子类虚函数可返回子类对象的指针或引用(协变), 这个返回的基类和子类可以是其他继承体系中的

函数重载, 隐藏和覆盖的区别?(经典面试题)

- (1) 首先, 覆盖是针对继承体系中的虚函数的, (把上面的内容拿下来), 而在继承体系中,
- (2) 父子类中有同名函数并且不构成覆盖, 那么它们就构成隐藏.
- (3) 对于函数重载, 是在同一作用域中出现同名函数, 在继承中的父子类不是在同一作用域

49. override 和 final 关键字的用处

在一个基类的虚函数后面加上 final, 代表此虚函数不能再被重写. 或此类无法被继承

在基类虚函数后加 override, 会强制检查子类有没有重写此虚函数, 如果没有重写就报错

50. 什么是纯虚函数? 什么是抽象类?

在虚函数后面写上=0, 这个虚函数就叫纯虚函数, 而包含纯虚函数的类叫做抽象类

抽象类无法实例化出对象, 派生类继承抽象类后, 必须重写纯虚函数才能实例化出对象

51. 聊聊多态的原理?

- A. 有虚函数的类对象中, 有一个虚函数表指针, 这个表中存放的虚函数的地址, 当子类没有重写父类虚函数时, 父子类的虚函数表中的虚函数的地址是一样的, 而当子类重写了父类的虚函数后, 子类会用新的地址覆盖父类虚函数的地址
- B. 调用虚函数时, 指针指向子类对象就调用子类虚表中的虚函数, 指向父类就调用父类虚表中的虚函数

52. 运行时多态和编译时多态?(动态多态和静态多态)

运行时多态: 是通过虚函数和继承来实现的, 是指在程序运行时去到对象中虚函数表指针指向的虚函数表中找函数地址, 运行时才知道调用谁

编译时多态: 通过函数重载和运算符重载实现, 是指程序在编译阶段根据函数名, 来确定调用函数的地址, 编译时就知道调用谁了

53. 为什么在继承体系中最好把析构函数定义为虚函数?

- A. **若不写为虚函数:** 父类指针指向的是子类对象时, 析构函数就是普通的函数调用, 是编译时决定, 指针是父类的就会调用父类的析构函数, 就可能会有问题
- B. **将析构写成虚函数后:** 由于父子类的析构函数都会被解释为 Destructer. 所以一旦写成虚函数就构成函数重写. 析构时是运行时决定, 指针指向父类就调用父类的析构, 指向子类就调用子类的析构

54. 虚函数可以是内联/构造/静态函数吗?

- A. 虚函数不能是内联函数, inline 函数没有地址, 是使用时直接展开, 无法存入虚表
- B. 静态成员不能是虚函数, 因为它不能通过 this 指针访问, 而虚函数恰恰要用 this 访问

C. 构造函数不能是虚函数, 因为对象中的虚函数表指针是在构造时才初始化的

55. 二叉树的前中后序遍历的非递归版本

A. 前序遍历:

```
while (cur != nullptr || !st.empty())
{
    while (cur != nullptr)
    {
        ret.push_back(cur->val);
        st.push(cur);
        cur = cur->left;
    }
    TreeNode* top = st.top();
    st.pop();
    cur = top->right;
}
```

B. 中序遍历:

```
while (cur || !st.empty())
{
    while (cur != nullptr)
    {
        st.push(cur);
        cur = cur->left;
    }
    TreeNode* top = st.top();
    ret.push_back(top->val);
    st.pop();
    cur = top->right;
}
```

C. 后序遍历:

```
while (cur || !st.empty())
{
    while (cur)
    {
        st.push(cur);
        cur = cur->left;
    }
    TreeNode* top = st.top();
    if (top->right == nullptr || top->right == prev)
    {
        ret.push_back(top->val);
        st.pop();
        prev = top;
    }
    else cur = top->right;
}
```

56. 了解过高度平衡二叉搜索数吗?

A. 高度平衡二叉搜索数就是 AVL 树, 它是为了解决普通二叉搜索树可能是单枝树从而导致查找效率低下的问题.

B. AVL 树的性质是当向 AVL 树中插入一个元素后, 要保证每个节点的左右子树高度差不超过 1, 若违反规则会旋转整个树重新达到平衡.

57. 了解过红黑树吗?

A. 红黑树简介:

红黑树是一颗二叉搜索树, 通过对任何一条从根到叶子的路径上的各个节点的着色限制, 达到最长路径不超过最短路径的二倍

B. 红黑树的性质:

- (1) 每个节点不是红色就是黑色
- (2) 根节点是黑色
- (3) 红色节点的两个孩子节点一定是黑色
- (4) 从一个节点到其任意后代叶子节点的路径上, 包含相同数量的黑色节点
- (5) 叶子节点是黑色(空节点是黑色)

58. 什么是哈希冲突?怎样解决?

A. 哈希冲突就是两个不同的关键字通过同一个哈希函数计算出相同的结果, 从而映射到哈希结构的同一个位置

B. 通常使用闭散列(开放地址法)或开散列(链地址法)解决哈希冲突:

- (1) **闭散列:** 当发生哈希冲突时, 若哈希表没有被填满就往后找空闲的位置放入(线性探测)
- (2) **开散列:** 通过哈希函数算出相同结果的数据放在同一个子集中, 每个子集称为桶, 各个桶中的元素通过单链表连接起来, 链表的头节点放在哈希数组中
- (3) **闭散列的缺陷:** 不同关键码可能占据了可利用的空位置, 使得寻找某关键码的位置需要许多次比较, 导致搜索效率降低, 应该使用链地址法

59. 给 40 亿个不重复的整数,如何快速判断一个数是否在这之中?

使用位图, 40 亿个整数只需要 40 亿个 bit 为来存储,也就是 5 亿字节 $\approx$ 500MB 的空间

60. 给一个超过 100G 的 log file,log 中存着 IP 地址,设计算法找到出现次数最多的 IP 地址?

使用哈希切分, 读取每一个 ip,利用哈希函数将所有的 ip 放入不同的小文件中, 相同的 ip 一定进入了相同的小文件, 再使用 `unordered_map<string,int>` 对每个小文件进行统计

61. 给两个文件,分别有 100 亿个请求,只有 1G 内存, 如何找两个文件交集?

使用相同的哈希函数将 A,B 文件都拆分为 1000 份小文件, 再使用两个 set 去编号相同的 A_i 和 B_i 的小文件找交集

62. C++11 的新特性你都了解哪些?

A. 先说自己会的东西:

自动类型推导, 范围 for, 智能指针, 右值引用, lambda 表达式, 线程库等

B. 再问问需要细谈哪个?下面把常问的都列举出来

C. 什么是右值?什么是左值?怎样区分?

左值就是可以出现在等号左边的值,如变量名, 而右值是只能出现在等号右边的值,比如字面常量, 表达式返回值等,是将亡值. 区别左值和右值最简单的方式就是是否可以取地址, 能取地址的就是左值, 不能取地址的就是右值

D. 右值引用的使用场景?有什么用?

在传函数参数时,如果传递的是右值,右值本身就是将亡值,再进行深拷贝代价太大了,用右值引用进行移动拷贝,直接把将亡值拿过来. 在容器中可以实现右值引用版本的移动构造和移动赋值函数,可以大大的增加效率

E. 了解过万能引用和完美转发吗?

模板中的&&是万能引用, 它既可以引用左值也可以引用右值。但在后续的使用中, 都会退化为左值, 想要解决这个问题需要使用 forward 完美转发在传参过程中保留原有属性

F. 谈一下 auto_ptr, unique_ptr, shared_ptr 和 weak_ptr?

- (1) auto_ptr 用 p2 拷贝构造 p1 后, p1(p2),p2 就被置空了
- (2) unique_ptr, 既然 auto_ptr 拷贝有问题,那我就无法进行拷贝构造
- (3) shared_ptr,采用引用计数的方式, 这是最正确的版本
- (4) Weak_ptr 是设计出来专门解决 shared_ptr 的循环引用的问题的

G. 什么是 shared_ptr 的循环引用问题?

两个或多个 shared_ptr 对象相互持有对方的引用, 形成一个闭环, 导致它们之间的引用计数永远无法归零, 从而造成内存泄漏。

H. 手撕一个 shared_ptr?

先不考虑定制删除器, 成员有:T* _ptr, int* _count, mutex* _pmtx. ++和--要加锁.

成员变量都在堆上申请空间, 析构时都要 delete

63. 了解过 C++14 和 C++17 新特性吗?

C++14:

1. Make_unique 用于初始化 unique_ptr
2. 标准库的拓展, 支持读写锁
3. Exchange 用于原子地交换一个对象的值
4. 字符串面值: R" (ABC/" /)"
5. 二进制面值: 0b01011101
6. 新增分隔符: b = 9' 1234' 5678 方便阅读

C++17:

1. If 和 switch 可直接初始化变量作为条件表达式的一部分

```
If(int* p = nullptr; !P){}
```

2. String_view: 轻量级的只读字符串. 占用内存更少

64. 什么是设计模式?手撕一个单例模式?

- A. 设计模式就是一套被大家反复使用的代码的经验总结
- B. 我了解单例模式, 一个类只能实例化出一个对象就是单例模式
- C. 手撕单例模式, 饿汉模式:程序启动就创建对象,懒汉模式:使用时才创建(需要加锁)

65. 了解过 C++ 的强制类型转换吗?

C++ 的类型转换有四个, static_cast, dynamic_cast, reinterpret_cast, const_cast

- (1) `static_cast`: 用于两个相关的类型进行转换, 不能用于父子类
- (2) `dynamic_cast`: 将父类对象的指针或引用转换为子类对象的指针或引用(只能用于父类有虚函数的类)
- (3) `reinterpret_cast`: 用于两种不相干的类型之间的转换
- (4) `const_cast`: 用于删除变量的 `const` 属性

Linux 系统部分:

66. 权限掩码有什么作用?知道粘滞位吗?

- A. 可通过设置权限掩码的方式来改变文件创建出来后的默认属性. 文件夹默认权限(775)
普通文件默认权限(664)
- B. 首先, 目录的读,写,可执行权限分别代表:是否可以查看目录下的文件, 是否能创建或删除目录下的文件, 能否进入此目录. 在同一个目录下工作的成员可以随意的删除别人的文件, 若把目录的 `w` 权限关闭,则也不能创建文件了, 所以给目录设置粘滞位后,用户只能删除自己的文件, 而不能删除别人的文件

67. 怎样将程序停留在预处理过程/编译/汇编过程后?

- (1) 预处理后: `gcc -E code.c` 形成.i 文件
- (2) 编译后: `gcc -S code.i` 形成.s 文件
- (3) 汇编后: `gcc -C code.s` 形成.o 文件

68. 用过 gdb 吗?讲讲常见的热键

- (1) `gdb +` 可执行程序进入调试模式, `quit` 退出调试模式

- (2) list(简写 l) 会将代码打印出来
- (3) run(简写 r),类似于 VS 下的 F5
- (4) Next(简写 n), 向下走一步, 类似于 VS 下的 F10
- (5) b+行号, 在任意行打断点, b+函数名,给函数打断点
- (6) Info+b 查看打的断点, d+断点编号, 删除对应断点

69. 谈谈你对整个计算机体系的认识

冯诺依曼体系:由输入设备, 输出设备, 内存和中央处理器构成. CPU 只能和内存进行交互, 数据不能由外设直接导入到中央处理器进行运算, 要先将数据搬到内存.

70. 什么是进程?谈谈你自己的理解?

- (1) 课本概念: 进程就是一个正在运行的程序
- (2) 内核观点: 进程是担任系统资源分配的实体
- (3) 自己的理解: 进程就是可执行程序+内核数据结构(PCB)

进程包括很多知识点, 比如进程状态,进行的调度优先级, 进程管理和文件管理, 进程管理和信号管理

71. 进程在运行时可能会出现哪些状态?

- A. 运行状态: 只要在运行队列中或正在 CPU 中运行就处于运行状态
- B. 阻塞状态: 进程在访问某些资源时, 若资源还没就绪,进程会被放在等待队列中, 就从运行状态变成了阻塞状态
- C. 挂起状态: 若一个进程处于阻塞状态,那么就代表它在一段时间内无法被调度, 恰好此时

OS 的内存资源已经严重不足, OS 会将这个进程对应的代码和数据暂时放到磁盘中, 为内存腾出空间, 这个进程就被挂起了

D. 进程状态的本质是 PCB 中的一个 status 字段, 进程状态改变的本质就是修改 PCB 中的字段并且将 PCB 链接到不同的队列中

72. Fork 函数了解多少?

Fork 函数用于创建子进程. 返回值大于 0 代表是父进程执行流. 返回值等于 0 代表是子进程执行流.

73. 了解过僵尸进程和孤儿进程吗?

A. **僵尸进程**: 当一个进程退出后, 退出信息还没有被父进程读取, 操作系统必然维护这个进程的 PCB 结构,此时这个进程处于僵尸状态

B. **孤儿进程**: 当子进程还没退出而父进程先退出了, 这个子进程就是孤儿进程, 孤儿进程会被 1 号进程领养

74. 并行和并发的区别?

A. 并行: 多个进程在多个 CPU 下同时进行运行

B. 并发: 多个进程在一个 CPU 下采用进程切换的方式, 在一段时间内多个进程都得以推进

75. 当发生进程切换后再次被调度时,怎样知道上次运行到哪儿了?

发生进程切换时, 会不断对自己的上下文进行保存和恢复, 进程切走时,会将寄存器的数据保存到 PCB 中, 而进程再次调度时, 会将 PCB 中的数据重新导入到内存. 程序计数器 eip

可以记录当前程序运行到哪个位置了.

76. 了解过哪些环境变量?

首先, 环境变量是指在操作系统内用于指定某些运行环境的参数. 我接触过的环境变量有

PATH: 指定命令的默认搜索路径. HOME: 用户的主工作目录. PWD: 当前的路径

77. 什么是地址空间? 和物理内存是什么关系? 为什么要有它?

- A. 本质是一种内核数据结构, 它至少包含了对各个区域的划分,和某个特定的进程相关联.
- B. 站在系统角度, 地址空间我把它理解为 OS 为每一个进程画的饼, 它是虚拟地址
- C. 虚拟地址通过页表映射到物理地址
- D. 虚拟地址和页表的存在保证了进程间的独立性, 每个进程都认为自己都 4GB 的空间.
- E. 代码申请和释放内存在虚拟地址中完成, 可以将物理内存的分配与进程的管理解耦
- F. 保证了物理内存的安全,凡是非法访问或映射都会在页表处被拦截下来

78. 谈谈你对整个文件系统的理解?

(1) **已被打开的文件(内存文件):** 操作系统为了方便管理内存中所有已打开的文件, 会构建 struct file 结构体,充当一个被打开的文件, 再用链表将所有已打开的文件连接起来.每个进程中有一个文件结构体数组, 代表此进程打开了哪些文件, fd 的本质就是此数组的下标

(2) **没有被打开的文件(磁盘文件):** 首先,磁盘分为盘面,磁道和扇区,每个分区会划分不同的块组,块组中有 inode 用来存储文件的属性, datablock 用来存储文件的内容,inode 位图来标记该 inode 是否被占用.目录文件会保存 inode 和文件名的映射关系,可以通过这个映射在磁盘上找到你想要访问的文件

79. 聊聊软硬链接?

- (1) 硬链接没有独立的 inode, 不是一个独立的文件,只是在目录下建立了文件名和指定 inode 的映射关系,相当于起别名, 创建硬链接时链接数目会增加
- (2) 软链接有独立的 inode,是一个独立的文件,存放另一个文件的路径名指向, 类似于快捷方式,创建软链接时链接数目不会增加

80. 动静态库的区别?

- (1) **静态库**: 程序在编译链接时就把库的代码链接到可执行文件中, 一旦程序跑起来,即使把静态库删除了也是没问题的,静态库以.a 结尾
- (2) **动态库**: 程序在运行的时候才去链接动态库的代码,多个程序共享使用, .so 结尾

81. 了解哪些进程间通信的方式?

- A. **匿名管道**: 单向的,一个用于读取,一个用于写入,通常用于父子进程
- B. **命名管道**: 调用 mkfifo 函数在当前目录下创建管道文件,也是单向通信的
- C. **共享内存**: 在物理内存申请一份空间,将此空间与对应进程的上下文进行映射到地址空间的共享区.两个进程使用相同的 key,就看见了相同的共享内存. 指令 ipcs -m 查看共享内存
- D. **信号量**: 本质是一个计数器,访问临界资源时,必须先申请信号量(sem--,预定资源),使用完毕后再 sem++,释放资源.PV 操作
- E. **信号**: 用于通知进程发生了某种事情, 常用于异步通信
- F. **它们的区别**: 共享内存进行通信是在进程的地址空间中属于内存的访问,这些数据的接收与传递不会涉及到内核,而管道的读写要使用 read 和 write 等系统调用进行通信,要涉及到内

核空间. 通信的本质都是不同的进程看见同一份资源

82. 聊聊你对整个信号体系的理解

A. **信号的概念:** 信号是进程之间的一种通知机制, 用户或 OS 通过发送信号通知进程某些事情已经发生,进程要在后续进行处理

B. **信号的产生:** 终端按键(CTRL C), 调用系统函数(kill, abort), 软件条件产生信号(管道的读端关闭会产生 SIGPIPE 信号), 硬件异常产生信号(指针错误,段错误)

C. **信号的保存:** 进程 PCB 中有 block(信号是否被阻塞),pending(信号是否在未决状态)两位图结构和 handler 方法表(信号的处理方法)

D. **信号的捕捉:** signal 函数可以自定义设置信号的捕捉方法. 从内核态返回用户态时可以进行信号的检测和处理,当发生中断, 异常或执行系统调用时,寄存器 CR3 中的权限会由用户态改为内核态,调用完系统调用后会回到用户态,若此时会进行信号的检测与处理

83. 了解过核心转储吗?

当进程出现某种异常时,是否把当前进程在内存中的核心数据转移到磁盘上,若信号对应的默认方法是 core 就会发生核心转储,使用 gdb 调式时输入 core file 可直接定位到出错的地方

84. 线程和进程的区别?

A. 进程是资源分配的基本单位,而线程是操作系统调度的基本单位.

B. 进程拥有独立的地址空间和页表,而线程则是共享进程的地址空间和其他资源

C. 进程的栈由地址空间维护,而线程的栈空间由线程库维护,映射到共享区

85. 了解过协程吗?

我将协程理解为轻量级线程, 它不会像进程和线程一样进行上下文的切换, 省去了 CPU 的切换开销. 并且它更轻量, 更容易实现高并发

86. 用过哪些锁?讲一讲死锁的原理

- A. 互斥锁,同一时间只允许一个线程获得这把锁并进入临界区,其他线程只能阻塞等待
- B. 读写锁,一次只有一个线程可以占有写模式的读写锁, 但是可以有多个线程同时占有读模式的读写锁.
- C. 死锁是指在一组进程中的各个线程均占有不会释放的资源,并且各个线程相互申请对方所占用的不会释放的资源而处于一种永久阻塞的状态

87. 讲讲线程安全和可重入的联系?

- A. 线程安全是指多个线程并发执行同一段代码时,会出现不同的结果.
- B. 重入指同一函数被不同执行流调用,当前流程还没执行完,其他流程就进入了.
- C. 可重入的函数是线程安全的,线程安全不一定可重入

88. 什么是线程同步?

- A. 在保证数据安全的前提下,让线程能按照某种特定的顺序访问临界资源,避免饥饿问题
- B. 让线程检测到临界资源不就绪时,不要让线程再频繁检测了,让线程等待.当资源就绪后,通知对应的线程再让它进行访问,使用条件变量解决上面的问题

89. 什么是生产者消费者模型?

生产者消费者模型的本质就是利用一个阻塞队列作为缓冲区,将生产者和消费者进行解耦.消费者不从生产者中拿数据,而是从缓冲区中拿,同样生产者直接将数据扔给缓冲区

90. 手撕一个线程池?

用队列存储要执行的任务,用 nums 表示要启动的线程数量,_mtx 互斥锁,_cond 条件变量实现 push,pop,routine(线程要执行的函数),initThreadPool(启动线程池,创建 nums 个线程)

91. 操作系统用户态和内核态有什么区别?

- A. 用户态是一种受限的状态,它只能访问有限的资源,并且执行有限的操作
- B. 内核态是具有完全的控制权和访问权的状态,执行代码时不会受限
- C. 当调用系统调用,程序发生异常时,会由用户态转变到内核态,在内核态的优先级会非常高
- D. 用户态和内核态相互切换的本质是,CR3 寄存器中的值从 3 变为 1

Linux 网络部分:

92. TCP 和 UDP 的区别?

- (1) TCP 是面向字节流的(不关心数据分几次发你,也不关心你几次读取完,只关心尽快将数据发给你),而 UDP 是面向数据报的(一次发送一个完整的报文,对方也接收到一个完整的)
- (2) TCP 通信前要先建立连接,而 UDP 不用
- (3) TCP 有确认应答,超时重发等机制保证可靠性,而 UDP 是不可靠的
- (4) TCP 的报头的大小是可变的,而 UDP 报头大小是固定的

93. 套接字编程一般的流程是什么?

- (1) 调用 socket 创建套接字
- (2) 使用 bind 将服务器的 IP 和端口号绑定到 OS 内部
- (3) Listen 进行监听
- (4) 用 Accept 接收客户端的请求
- (5) 客户端使用 connect 发起连接

94. Cookie,session 和 token 有什么区别?

- (1) Cookie 是一段保存在客户端本地的数据,用来保存用户的个人信息,服务器通过 cookie 来区分不同的用户,,此信息可以被伪造
- (2) Session 是存储在服务器上的数据,服务器会根据客户端发来的 sessionid 来确认是哪个会话,无法伪造
- (3) token 是一段加密的字符串, 包含的用户信息, 通过请求头传递. 适合在分布式系统中使用.

95. HTTP 和 HTTPS 的区别?

HTTPS 会进行对称,非对称加密和证书认证的方式使其比 HTTP 安全许多

96. 讲一下 TCP 的三次握手和四次挥手?

- (1) 三次握手的目的是为了建立连接,首先客户端会向服务器发送一个包含 SYN(请求连接)的 TCP 报文,而后服务器收到这个请求后会向客户端发送包含 SYN 和 ACK(确认收到)的 TCP

报文,最后,客户端向服务器发送包含 ACK 的 TCP 报文后建立连接

(2) **四次挥手的目的是为了断开连接**,客户端会先向服务器发送一个包含 FIN(结束连接)的 TCP 报文,服务器在给客户端发送一个包含 ACK 的 TCP 报文后,立马发送 FIN,最后客户端再向服务器发送 ACK 后进入 TIME_WAIT 状态

(3) **为什么是三次握手?不是一次两次?** 首先,最后发送报文一方会先建立连接,而另外一方会收到 ACK 确认后才会建立连接,如果是偶数次握手,那么客户端就可以无脑向服务器发送连接请求,并且忽略掉服务器发来的 ACK,这样就可以低成本攻击一个服务器.所以握手的次数需要是奇数.一次握手也是如此.而三次握手就能达到目的,也就不需要五次或七次握手了

(4) **什么是 TIME_WAIT 状态?** 主动断开连接的那方要维持一段时间的 TIME_WAIT 状态,保证对端收到了最后一个 ACK 报文(对端可能会超时重发 FIN).该状态下连接已经终止,但 ip 和端口号依旧被占用

97. TCP 通过什么来保证可靠性?

- (1) **确认应答**: 保证历史数据都收到了
- (2) **超时重发**: 若发送数据后一段时间没有收到确认应答,会重发
- (3) **流量控制**: 发送端会根据接收端的接收能力来决定发送速度(对方窗口大小)
- (4) **拥塞控制**: 关心当前网络的状态,利用慢启动机制窥探网络状态

98. TCP 通过什么方式来提高性能?

- (1) **滑动窗口**: 就是在对面来得及接收的前提下(对方窗口大小),一次性发送多条数据,这么多数据只会收到一个 ACK,提高效率.滑动窗口的本质就是缓冲区的两个下标.超时重传中,重传的数据也保存在滑动窗口内

(2) **延迟应答:** 不能收到数据就返回 ACK,自己的窗口可能比较小,应该每隔 N 个包应答一次

99. 了解哪些 TCP 协议的报头数据?

- (1) 源端口号和目的端口号
- (2) 序号和确认序号
- (3) 四位首部长度和六位标记位
- (4) 窗口大小: 接收缓冲区的大小

100. 如何解决 TCP 的粘包问题?

- A. 使用定长的包,每次都按固定大小读取即可
- B. 对于变长的包,可以约定在最前面的字段代表包的总长度
- C. 对于变长的包,可以在包和包之间加分隔符

101. 什么是 IP 协议?

IP 协议是一种网络层协议,它负责将数据包可靠的从源主机跨网络传输到目标主机

102. IP 协议的字段了解多少?

- (1) 源 ip 地址和目的 ip 地址
- (2) Ip 协议的版本号和上层的协议类型
- (3) 16 位标志,若报文在数据链路层被分片了,属于同一个报文的片中标志相同
- (4) 13 位片偏移,标识分片相对于原始报文的偏移量

103. 什么是网段划分?

网段划分是将 ip 地址划分为网络号和主机号,相同网段中的主机的网络号相同.可以通过将 ip 地址按位与上子网掩码来得到网络号

104. WAN 口 ip 和 LAN 口 ip 的作用?

- (1) 路由器 LAN 口 IP 连接的主机, 都从属于当前这个路由器的子网中
- (2) WAN 口 IP 就是一个公网 IP 是自己所在的上级子网给自己分配的,子网内的主机需要和外网进行通信时, 路由器将 IP 首部中的 IP 地址替换为 WAN 口 IP

105. 了解过 NAT 技术吗?

子网内的主机需要和外网进行通信时, 路由器将 IP 首部中的 IP 地址进行替换(替换成 WAN 口 IP), 这样逐级 替换, 最终数据包中的 IP 地址成为一个公网 IP. 这种技术称为 NAT

106. 谈一谈你对 ARP 协议的理解

ARP 协议建立了主机的 ip 地址和 mac 地址的映射关系.通过广播的方式让对方主机将自己的 mac 地址信息应答回来

107. 有了 ip 地址为什么还要有 mac 地址?

- 1. MAC 地址可以唯一标识同一网络段或广播域中的主机。这是实现帧转发的基础。
- 2. 如果只使用 IP 地址,所有的路由都需要在网络层进行,效率会降低.而 mac 地址可以直接在链路层进行路由,效率较高

3. 当一个 IP 地址被多个主机共享使用时(通过 NAT 等技术),MAC 地址可以唯一确定主机的物理信息

108. 了解过 MTU 吗?

MTU 是以太网传输的最大单元,一般为 1500 字节,超过这一大小会进行数据分片

109. 在浏览器输入一段 URI 后拿到资源的整个过程?

- (1) 首先会进行 DNS 域名解析拿到服务器的 ip 地址.
- (2) 然后双方通过 TCP 的三次握手建立连接.
- (3) 浏览器构建好请求后向服务器发送 HTTP 请求
- (4) 服务器将请求读取到应用层后解析请求的方法和资源等
- (5) 服务器定位到资源后构建 HTTP 响应将此响应通过网络再推送回去
- (6) 最后浏览器获取到资源后双方进行四次挥手断开连接

110. 讲讲 select,poll,epoll 的优缺点

- (1) 所有的多路转接方案都具备效率高,可以同时处理多个连接的优点
- (2) **select**: 编码比较复杂,需要维护一个第三方数组来保存所有合法的 fd,充斥着大量的遍历,并且每次都要对 select 的输出型参数进行重新设定,同时管理的 fd 个数有上限
- (3) **poll**: 输入输出型参数是分离的,不用大量的重置,可同时管理的 fd 个数没有上限.但 poll 也需要不少的遍历
- (4) **epoll**: 编码比起前面两个更简单,避免了使用遍历机制,通过回调函数的方法将就绪的文件描述符结构加入到就绪队列中,上层只需访问就绪队列即可.类似于生产者消费者模型

111. Select/epoll 编程一般的代码模式

(1) **Select 编程步骤:** a. 需要一个第三方数组用来保存所有合法的 fd. b. 遍历数组,更新出 fd 的最大值,并且将需要关心的事件添加到 fd_set 位图中. c. 调用 select 进行事件检测. d. 再遍历数组,找到就绪的事件,完成对应的动作

112. 水平触发和边缘触发的区别?

(1) 水平触发(LT 模式)当 epoll 检测到 socket 事件就绪时可不立刻进行处理,或只处理一部分,因为 LT 模式需要把缓冲区上所有的数据都读完后 epoll_wait 才不会立刻返回

(2) 边缘触发(ET 模式)当 epoll 检测到 socket 上事件就绪时必须立刻读取,并且要读取完.ET 模式更加高效,通知用户数据就绪的次数少了就减少了从内核态到用户态的转换,站在 TCP 协议的角度,ET 模式会倒逼程序员尽快将数据取走,从而给对方发送一个更大的接收窗口,对方就可以有更大的滑动窗口一次性发送更多数据,提高了 IO 的吞吐量

113. 为什么 ET 模型必须用非阻塞读取?

ET 模式只会通知你一次,有数据到来. 在 ET 模式下来了个就绪事件后, 我们必须把数据一次性从接受缓冲区读取完. 使用 recv 函数时一定是循环读取, 若文件描述符是阻塞的, 那么最后一次 recv 就会被阻塞, 直到下一次此连接的事件到来, 当然此连接可能未来不会再来数据, 就会一直阻塞在这里

114. Sock 编程中,read 和 recv 函数有什么区别? write 和 send 呢?

Read 和 recv 都可以用于获取某个 fd 的数据, 但是, read 是标准的 IO 函数, 它可用于网络的原因是套接字本质也是一直文件描述符, 而 recv 是 socket 编程中的 API, 它提供了 flag

选项. send 和 write 也是如此

排序算法部分:

115. 了解过哪些排序算法?它们的时间复杂度和稳定性如何?

- (1) 冒泡排序, $O(N^2)$, 稳定的
- (2) 插入排序, $O(N^2)$, 稳定的
- (3) 选择排序, $O(N^2)$, 不稳定
- (4) 希尔排序, $O(N \cdot \log N)$, 不稳定
- (5) 堆排序, $O(N \cdot \log N)$, 不稳定
- (6) 快速排序, $O(N \cdot \log N)$, 不稳定
- (7) 归并排序, $O(N \cdot \log N)$, 稳定

116. 手撕各种排序算法

数据库部分:

117. 介绍一下 MySQL 中的 curd 操作?

主要涉及的是增删查改操作

- A. 增加数据: insert into
- B. 替换数据: replace into
- C. 查看数据: select 系列语句, 可以通过 where 来筛选要查找的条件, 可以使用 order by 来对数据进行排序 (ASC 为升序, desc 是降序), 也可以使用 group by 先对整张表进行

分组后再查询

D. 更新数据: update 表名 set 列名 = ...

E. 删除数据: delete 配合 where 使用

F. 筛选分页结果: 使用 limit s, n

118. SQL 查询中各个关键字执行的先后顺序

From > where > group by > having > select > distinct > order by > limit

119. innodb 和 myisam 有什么区别?

1. 事务支持: innodb 支持事务, 但 myisam 不支持
2. 外键支持: innodb 支持多个表之间建立关联关系. myisam 没有
3. 索引结构: innodb 是聚簇索引, myisam 是非聚簇索引.
4. Count(*)的效率: innodb 不保存具体的行数, 每次取 select 行数时, 会全表扫描.

myisam 用了一个变量来保存行数, select 时只需输出变量的值即可

120. 什么是 MVCC?

即多版本并发控制, 是一种数据库并发控制的技术。它通过维护数据的多个版本来实现读写

操作的隔离, 从而提高数据库的并发性能和可靠性

121. 讲一下 MySQL 的索引?

A. 索引的本质: 是一种用空间换时间的策略, 类似于书本的目录,可快捷定位

B. 索引做了什么事情: MySQL 中的数据以 page(16KB)为单位进行保存的. 但是这 16KB 不

全用来存储数据,部分空间用于存储目录,标识主键的位置.

C. 索引的底层: 索引的本质就是一颗 B+树, 它的非叶子节点不存储数据, 类似于书籍的目录, 而叶子节点存储的是数据, 这棵树的特点是矮并且胖, 所以找到目标数据的 IO 次数会减少, 可以提高效率。(innodb 的叶子节点存储数据, MYISAM 的叶子节点存储数据的地址)。

D. 索引的分类: MySQL 中: 聚簇索引, 非聚簇索引. 其他数据库: 稀疏索引, 跳表索引

122. B 树和 B+树的区别?为什么 MySQL 用 B+树做索引?

1. 数据的存储方式: B 树的非叶子节点也叶子节点都会存储数据, 而 B+树只会在叶子节点存储数据, 非叶子节点存储索引信息
2. 空间利用率: B 树每个节点都会存储关键字, 使得此结构需要事先预留空间来存储数据和指针, 而 B+树只有叶子节点有数据, 空间利用率高
3. 查找效率: B+树只有叶子节点存储数据, 并且数据有序的通过链表连接, 查询效率高, B 树需要在非叶子节点和叶子节点都执行查询

123. 讲一下 MySQL 的事务?

- A. 什么是事务: 是一组 DML 语句, 一个事务中的所有语句被看作一个整体, 这些语句要么全部执行成功, 要么一条都不执行
- B. 有什么用: 事务的存在在上层看来, 完成某个任务的动作是原子的, 不需要程序员去考虑潜在错误和并发问题, 事务本质上是为应用层服务的
- C. 事务的特性: 1. 原子性, 一个事务中的语句要么全部执行, 要么都不执行, 在中间某个环节发送错误会进行回滚。2. 持久性: 一旦一个事务处理结束, 那么在这个事务中对数据

的修改就是永久的，即使系统崩溃也不会丢失。3. 隔离性：隔离可以防止多个事务并发执行时可能会导致数据不一致的问题（虽然说在上层看来事务是原子的操作，但在事务执行内部是有多条语句的）。事务分为四个隔离级别：1. 读未提交：所有事务都可以看见别的事务还没有提交的数据，会出现脏读，不可重复读等问题，基本上不使用。2. 读提交：一个事务只能看见别的已提交的事务的数据，可能会出现不可重复读问题。3. 可重复读：指在一个事务的执行周期中，多次读取数据会看见同样的结果。（会有幻读，看见别的事务的 insert 操作,但 MySQL 不会）4. 串行化：强制事务排序，使其不可能发生冲突。4. 一致性：在执行事务前以及结束事务后，数据库的完整性没有遭到破坏

124. 讲一下事务的四个隔离级别的实现原理？

1. 隔离级别是通过 MVCC 实现的，对于 MVCC，我想说三个点：表的三个隐藏字段，undo_log 和 read_view
2. 表中有三个隐藏字段：最近修改该记录的事务 ID，隐式主键和回滚指针，这三个字段会在后面的 undo_log 中用到
3. Undo_log 本质上就是一段 MySQL 维护的内存缓冲区，数据在修改前会被先记录到 undo_log 中
4. Read_view 是事务在执行快照读时，生成的一个数据结构，它其中包含了一些字段：高水位，低水位，当前活跃的事务 ID 集合，RR(可重复读)与 RC(读提交)的本质区别就是，可重复读不管进行多少次快照读，使用的 read_view 是一样的，而读提交中每一次进行快照读都会生成一份新的 read_view，来更新两段快照读之间已提交的事务

实习内容部分:

125. 用户资料的平台化建设详解

1. **项目介绍:** 从零实现了一个用户资料平台, 此平台提供用户资料的查询等周边功能
2. **项目背景:** 老的用户资料平台功能单一, 页面分散, 并且停止维护了, 最重要的是老的页面没有做权限管控, 每一次资料查询都会消耗我们组的人力资源
3. **项目大致做了什么事:** 此项目的本质工作就是在做三件事: (1) 前后端交互 (2) 后端与数据库做交互(权限管控) (3) 微服务间调用(完成用户需求)
4. **项目难点:** (1) 权限划分的细腻度问题 (2) 使用 protobuf 时, 元数据的动态性问题 (3) 面对海量 QQ 号查询时的 CPU 资源分配问题
5. **项目的收获以及成绩:** 目前此项目已经在公司内部全面投入使用了, 大大的减少了我们小组的人力支出. 个人收获: 我觉得最大的收获是进行团队合作和沟通时给我带来的满足感, 此平台多为其他部门的同学在用, 我要做成什么样子需要向它们进行调研, 这个过程当中我找到了自己的存在感和认同感, 并不像大家所说的实习生都是打杂的, 我觉得我做的事是有意

前言: 我们组是做用户资料关系链的, 所有用户的敏感信息, 修改记录以及好友列表都是我们组提供的服务. 其他部门经常会查询用户的某些字段来完成某种任务

项目背景: 1. 老的用户信息查询功能很单一, 并且页面分散, 使用起来不方便. 2. 老的页面没有做权限管控, 每一次其他部门的查询都会浪费我们组的人力资源. 3. 老的页面停止维护了, 并且老的代码五花八门, 不统一. 结论: 急需将用户资料门户相关的功能集成到一个平台, 解放生产力!

项目实现: 1. 分为主服务和异步服务. 主服务: 既要负责对接前端的所有操作, 又要对接 AI 智能体的请求, 所以在项目架构设计的初期, 主服务用于处理同步任务, 并且给异步服务派发耗时久的任务. 2. 项目的本质就是在做权限管控和数据处理. 并且其中充斥着大量的微服务调用和与数据库的交互.

代码实现: 1. 这个项目需要考虑未来的可拓展性问题. 所以代码的设计必须要有可拓展性, 必须要写的优雅. 2. 设计上采用了以分层组件的形式, ao 层处理业务层逻辑, 存放 handler 方法, repo 层处理具体的方法实现和数据处理. dao 层用设计模式抽象出了与 MySQL 和本地文件交互的方法. entity 层存放全局变量以及收发包的结构体实体

项目的难点: 1. 由于每一次微服务调用查询用户信息使用到的 protobuf 的元数据都不同, 并且字段一共有上千个, 还在不断更新, 所以不能直接定义 .proto 文件来进行调用. 需要使用到 protobuf 的反射, 在程序运行时动态的构建 .proto 文件. 2. 面对海量的 QQ 号查询时, 一次查询可能会花费几小时, 需要设计一个异步服务, 来专门针对这种场景. 不仅如此, 由于一次查询的时间过久, 必须要考虑容错性, 若查询失败或发生特殊情况, 异步服务需要保证数据的一致性, 要做容错处理 3. 微服务之间调用发的是 UDP 的包, UDP 的包是有大小限制的, 查询时一次性放太多 uin 会超过包大小限制, 而一次性放少量 uin 又会降低查询效率. 所以最后采用了每次放适量的 uin, 并且对一次调用没有被处理的 uin 做特殊处理 4. 权限的细粒度划分应该怎样进行? 往往一个小组需要查询的字段是相似的, 是以小组为单位划分权限, 还是以个人为单位划分权限? 为了后续项目的对接方便, 这里是以个人为单位进行权限管控

126. 42 亿流水记录的云迁移详解

1. 项目介绍: 将 42 亿条 QQ 用户的资料修改记录, 从本地分布式 MySQL 迁移到云数据库

click house 中

2. 项目背景: 老的分布式数据是存储在几台本地机器上的, 随着数据量增加, 不仅需要不断的增加外挂磁盘, 并且查询效率会显著下降. 并且一旦一台机器挂掉了, 运维成本非常大!

3. 项目大致做了什么事: (1) 产品选择(为什么要选择 click house, 不选择其他的数据库?) (2) 代码重构(根据 click house 的特性, 重写代码) (3) 存储结构优化(通过建表的方式来优化查询效率和数据压缩效率) (4) 灰度测试, 计算 CPU 等资源

4. 项目难点: (1) 考虑到存储痛点和成本问题, 如何选择产品 (2) 如何做到每分钟处理上万个请求(写入 2MB 数据), 但写机器数少 (3) 如何做数据级的容灾? (4) 如何优化 click house 的压缩效率?

5. 项目的收获以及成绩: **成绩:** 数据压缩的更小的(从原先的 18TB 减少到了 2TB), 成本更低了, 查询效率提升了. **收获:** 高性能服务器的设计理念, 对海量数据的存储和查询的思路, 对产品的调研能力

项目背景: 1. QQ 用户资料的所有修改记录, 有 42 亿条数据, 都在老的分布式 MySQL 数据库中, 老的存储结构是在五台机器的本地 MySQL 做的分布式策略, (1) 一旦挂了一台机器, 这台机器上的数据想要恢复就很麻烦, (2) 并且挂了一台机器后, 其他机器上的负载也会加剧. (3) 只有五台机器来存储, 前期数据量不大时查询效率还行, 随着数据量增多, 查询效率会很慢, 并且不能通过增加机器的方式来缓和 (4) 老的代码历史包裹重, 是纯 C 语言写的, 有些函数在新的 C++ 标准已经弃用了. 老的功能可视化做的不好. 数据云迁移后会做一个页面进行操作

产品选择: 1. 我们的项目的痛点主要是海量数据存储的问题, 查询和插入的效率慢是可以容忍的, 所以在选择产品时, 要以存储方案和性价比为主. 2. 在选择产品时我也有自己的思考, 虽然说所有的产品都会在内部腾讯云中申请, 但是也有成本考量, 所以这里摒弃了

tdsql 和 MySQL, 因为它们的优势在我们的项目中并不能很好的使用. postgresql 对数据存储并没有做优化, 并且成本较高. 所以这里选择的是列式存储结构, click house, 它有数据压缩和冷盘处理等操作, 很适用于我们的数据存储的痛点

项目规划: 1. 主服务负责接受新插入的 LOG 请求, 将信息写入共享内存队列中, 异步服务不断的从共享内存队列中拿数据, 然后写到数据库中. 这一整套服务视为一个整体, 把这个整体部署到多个机器上, 负载均衡式的处理新插入的 LOG 信息(每天大概有 33G 的 LOG 信息)

项目痛点: 1. 容错性, 将数据存储五个本地的 MySQL 服务上, 一旦一台机器挂了, 想要恢复数据需要将外挂磁盘换到另外一个机器, 并且需要重新配置文件路径, 运维成本太高
2. 存储问题, 一共只有五台机器来存储数据, 一旦容量过载, 不仅需要不断的增加外挂磁盘, 并且查询数据的效率也会变慢. 不仅如此, 存储占用越多, 成本就越大

项目难点: 1. 怎样做到大量的写操作, 但是机器数少? 每台机器每分钟会收到上万个插入 LOG 的请求, 一旦处理不过来, 很容易丢失数据. 解决方案: 采用主服务只保存数据到共享内存队列中, 而异步进程去共享内存队列中拿数据处理. 这里使用共享内存队列而不是直接写入文件的原因是, 使用共享内存是在内存层面直接做交互, 不会涉及到系统, 效率更高.

2. 怎样保证数据的安全性? 因为重要的数据都保存在内存中, 一旦进程挂掉了, 数据也就丢失了, 针对这种场景, 异步进程不能直接处理共享内存中的数据, 而是必须先将内存中的数据 dump 出磁盘后, 再去磁盘中拿数据处理, 以此来保证数据的安全性. 3. 产品选择, 市面上有很多产品可以供我们选择, 像主流的 MySQL, postgresql, MongoDB, SQL server 等数据库, 性能和可拓展性都不错, 我们选择了相对比较冷门的 click house 的主要原因是因为在同样的存储容量下, MySQL 的成本是 click house 的二倍, 它成本低, 并且 click house 作为列式存储结构, 会做数据压缩和冷盘处理, 这恰恰可以解决我们项目中的容量痛

点. **4. 怎样优化 click house 的压缩效率?** 将新老数据从两列变成一列, 因为新老数据的重
复度很高, 所以合并后可以压缩的更小. 并且数据的存储格式用 PB 比用 json 更优 **5. 怎样
优化查询效率?** 由于这个项目在使用使用, 会查询具体 QQ 号在一段时间内的修改记录. 所
以 QQ 号和日期是影响效率的关键因素. 所以这里按照天数作为分区, 并且用 QQ 号加修改
时间的方式作主键. click house 的主键是用于稀疏索引的, 每 8192 行数据会建立一个索引
条目. 但是这里我们的数据条数很多, 索引条目也会很多, 所以在稀疏索引的基础上, 我们
添加了一个跳表索引, 每个跳表索引会存储 128 个稀疏索引, 这样就大大的优化了我们的效
率

127. 什么是 protobuf?

Protobuf 本质上一种序列化协议, 类似于 json,xml. Protobuf 序列化后体积很小, 并且支
持多种语言, 适合在 RPC 调用时使用. protobuf 在使用时只需定义 .proto 文件, 系统会自
动生成源代码. protobuf 他的数据来源可以是 .proto 文件, 也可以是利用反射在程序运行
时动态的构建

128. 为什么 protobuf 比 json 和 xml 快?

Protobuf 采用二进制编码, 序列化和反序列化速度更快, 并且 protobuf 序列化后体积会比
json 更小

129. 什么是 RCP 调用?

是一种远程调用, 它可以像调用本地方法一样去调用远程的方法. 一般用于微服务间调用.

130. RPC 调用的底层原理是什么?

本质上就是一种网络调用, 请求方会在 stub 层构建函数的对应参数, 对端拿到请求后, 会解析包, 然后找到并调用对应的函数, 函数执行完后, 将结果再打包返还给请求方

131. 为什么有了 HTTP 还要有 RPC?

1. 性能方面: RPC 采用更高效的二进制编码形式传输数据, 而 HTTP 是基于文本的传输. 很多 RPC 框架支持连接复用, 在微服务架构中 RPC 可以抗住更大的并发量
2. 适用场景: 在公司内部系统中, RPC 可以提供更高效, 更可靠的通信方式, 并且在微服务架构中, RPC 可以适应微服务间频繁的调用.

132. 什么是消息队列?

消息队列就是一个存在于通信双方的中间件, 它用于异步解耦, 削峰填谷. 比较常用的消息队列有 Kafka

133. 你对 Kafka 了解多少?

Kafka 的本质是一种消息队列, 它是一个中间服务. 它有着高性能, 高可用, 容错高等特性.

1. 高性能: Kafka 内部会按照数据的不同类型, 分为多个 topic, 并且在每一个 topic 中, 会将队列进行分区, 分为多个 partition, 每一个 partition 可对应一个消费者
2. 高可用: Kafka 内部分为 leader 和 follow 两个服务, leader 服务 PC 交互数据, 而 follow 负责同步 leader 的数据, 当 leader 挂了, 就选一个 follow 顶上
3. 容错性: 假如所有的服务都挂了也没关系, Kafka 会做持久化策略, 会将数据写入磁盘.

Kafka 的零拷贝用的 sendfile, RockMQ 用的是 mmap

134. 什么是分布式系统?

将不同的功能分布到不同的服务器上, 将这些节点用网络连接, 组成一个完整的功能

135. 什么是负载均衡技术?

就是将大量请求或负荷, 平衡分配到不同的服务器上, 确保这些服务器能高效的处理任务

136. 什么是容器化技术?

将可执行程序及其配置打包到一个容器中, 使其可以在任意操作系统和平台上进行部署并运行

137. 什么是无锁队列?

是一种数据结构, 在并发编程中, 它不依赖于传统的锁机制, 而是利用原子操作, 内存屏障等方式, 实现线程安全. 优点: 减少阻塞, 减少上下文切换的开销, 提升效率

138. 什么是内存屏障?

内存屏障是一种为了防止编译器或 CPU 重排指令的技术,

139. 有哪些数据级容灾方案?

- (1) 持久化策略: 将数据 dump 到本地文件
- (2) 数据快照: 定期记录数据的最新状态, 发送故障时回滚
- (3) 灾备中心: 作为主服务数据的备份中心, 当主服务出现故障时, 可以切换到备灾中心

140. 什么是并查集?

概念: 就是一种将 N 个不同的元素划分成一些不相交的集合的数据结构, 通常用数组实现

实现: 组长节点存储集合内组员个数(负数), 组员节点存储组长的数组下标

用处: 可以快速的判断, 两个元素是否在同一集合中, 在图论的遍历算法中需要使用到

141. 什么是 B 树?

B 树本质上就是一种搜索结构, 常用于外搜索, 也就是磁盘上的搜索. 它是一颗平衡的 M 路搜索树, 它的节点是两个数组结构, 一个数组存储关键字(有序的), 另外一个数组存储的是孩子节点的指针. 它是一颗矮胖树, 它用于查找时, 在磁盘中的 IO 次数会很少, 会大大提升查找效率. 在我看来, B 树就类似于书本上的目录, 可以快速定位到你要查找的数据的位置

142. 什么是 B+ 树?

B+树是基于 B 树做的优化, 它的非叶子节点是不存储数据的, 类似于目录, 叶子节点是由有序链表组成的数据集合. B+树常用于做索引

143. 谈一下你对图的理解

1. 图本质就是由一些顶点和边组成的一种数据结构
2. 图的存储结构: 邻接矩阵: 节点的连通关系用二维数组存储, 邻接表: 节点的连通关系用链表存储
3. 遍历手段: 广度优先遍历: 先遍历完这个节点的所有相邻的顶点. 深度优先遍历: 先遍历完这个节点的某个单支路径
4. 最小生成树问题: 使用权重值最小的方式将图连通起来.

5. 求最小生成树的算法: **kruskal 算法**: 利用堆, 不断选权值最小的边, 再利用并查集, 选中后不构成回路就可选. **prim 算法**: 分 x 集合, 已被选中的点和 y 集合, 未被选中的点. 在 X 集合中选权值最小的边, 并且边的另一头在 Y 集合中

6. 最短路径问题: **Dijkstra 算法**: 本质上是一种贪心策略, 首先存在两个集合: S 和 Q, 先在 Q 集合中选择最短的路径, 将最短路径对应的点加入 S 集合, 然后反复如此去更新, 直到所有点都在 S 集合当中. **bellman-Ford 算法**:

144. 什么是跳表?

跳表是基于有序链表改良而来的, 它的每一个节点有随机层, 每一层都指向下一个同层节点, 这样遍历时, 就不用挨个遍历了. 跳表的平均时间复杂度是 $O(\log n)$. 通过概率 P 和最大层数 maxlevel 来控制整个跳表的结构

145. 跳表比起哈希和红黑树, 有哪些不同?

跳表的优势: 1. 遍历数据是有序的 2. 跳表相对于其他两个结果, 实现上更简单. 3. 跳表占用的空间比较小, 红黑树是三叉链结构, 并且有颜色和平衡因子等参数.

146. 什么是 LRU cache?

就是一种缓存算法, 缓存中保存最近最常使用的数据, 缓存满后会将最近最少使用的数据剔除出去

147. MySQL, postgresql, click house, MongoDB 有什么区别?

Postgresql 和 MySQL 一样是关系式数据库, 但是 postgresql 功能更强大, 涉及到高安全

性和复杂查询的场景时, 建议选择 postgresql

click house 是列式存储的关系式数据库. 它底层的引擎采用的是 merge tree, 它有分区, 稀疏索引, 跳表索引, 排序等操作, 使得查询效率很快. 并且 click house 内存存储时会用各种算法进行压缩,使得数据存量更小

MongoDB 是非关系式数据库, 它采用的是文档存储的形式, 文档中存储的是类似于 json 的数据类型. 如果项目需要灵活的数据模型,MongoDB 是个不错的选择

148. 什么是 redis?

Redis 本质上就是一个基于内存操作的, 键值类型的 nosql 数据库

它在服务和数据库之间的一层缓存, 常用于存储只读的热点数据. 它可以减轻数据库的并发压力

149. 什么是缓存击穿?

缓存击穿是指在高并发环境下, 大量请求同时访问缓存中不存在的数据, 导致这些请求穿透到数据库. 由于数据库无法承受如此高的并发访问, 可能会导致数据库压力骤增, 甚至引起数据库宕机。

150. 什么是缓存雪崩?

缓存雪崩是指缓存中的数据大量同时过期, 导致瞬间大量的请求直接打到了数据库, 可能会导致数据库瞬间压力过大甚至宕机。

151. 什么是缓存穿透?

缓存穿透是指查询一个一定不存在的数据, 由于缓存是不命中时需要从数据库查询, 而查询的结果在数据库中也不存在, 因此不会写入缓存. 这将导致这个不存在的数据每次请求都要

到数据库去查询，进而给数据库带来压力。

152. 什么是工厂模式？

工厂模式是一种设计模式，它通过定义一个创建对象的接口，但让子类决定要实例化的类是哪一个。工厂方法让类的实例化推迟到子类中进行。(对拓展开发,对修改关闭)

```
// 形状接口
class Shape {
public:
    virtual void draw() = 0;
    virtual ~Shape() {}
};

// 具体形状类：圆形
class Circle : public Shape {
public:
    void draw() override {
        std::cout << "Inside Circle::draw() method." << std::endl;
    }
};

// 具体形状类：方形
class Rectangle : public Shape {
public:
    void draw() override {
        std::cout << "Inside Rectangle::draw() method." << std::endl;
    }
};
```

153. 什么是观察者模式？

它定义了一种一对多的依赖关系，让多个观察者对象同时监听某一个主题对象。这个主题对象在状态发生变化时，会通知所有观察者对象，使它们能够自动更新自己。(如购物系统的订单状态改变时，不同的对象应该收到不同的内容)

154. 什么是 ES？什么时候会用到它？

ES 是一个开源的搜索引擎。在面对大规模数据的搜索和分析时，经常会使用到它，比如电商商城的关键搜索。ES 搜索效率高效的原因：倒排索引，分布式结构

155. 什么是 LSM tree?

是一种为了提高写入性能而设计的数据结构, 常用于数据库和存储系统中.

LSM tree 是日志写入式的, 并不是磁盘随机写入的,可大大提写入效率

156. 什么是 AIO?

即异步 IO. 不用阻塞等待操作完成, 能够更好的利用系统资源, 提高并发处理的能力.

AIO 对比同步 IO 的优势: 同步 IO 在程序等待 IO 时会浪费大量的时间, 而 AIO 可以在后台进行 IO 操作, 让程序同时处理多个任务

使用场景: 高并发服务器, 海量数据处理

实现方案: glibc 的 POSIX AIO 有标准接口

157. 高并发服务器的设计通常会使用哪些技术?

1. 负载均衡
2. 分布式架构
3. 异步 IO
4. 缓存技术: 减少对数据库的访问

158. 什么时候用 epoll?什么时候用 AIO?

1. Web 服务器,及时通信服务器,这种同步通信的场景中,用 epoll
2. 对复杂的业务逻辑, 对 IO 延迟不敏感,但追求高吞吐量的场景用 AIO

159. 什么是 epoll 的惊群效应?怎样解决?

1. 当底层有数据就绪时,会唤醒多个执行流,但实际上只有一个执行流会能处理此事件,其他执行流被唤醒会浪费 CPU 资源
2. 解决方案: 使用单线程或线程池来监听, 使用互斥锁

160. TCP 通信时,连接池满了再来请求会发生什么?

客户端往往会收到一个 RST(reset)报文, 也就是请求被拒绝, 或直接连接超时

161. HTTP, HTTP2.0, HTTP3.0 有什么区别?

1. HTTP: a. 对于每一个请求都要建立一个连接, 浪费资源 b. 纯文本传输,传输数据量大,效率低 c. 头部信息很冗杂有很多不必要的字段和重复信息
2. HTTP2.0: a. 多路复用, 可以在一个连接上同时发送多个请求, 提高资源利用率和性能
b. 二进制分帧,